

Distributed KV Cache Offloading for Mobile LLM Inference

A Simulation Study of Async Prefetch Over Wi-Fi 6

Data Communication Course Project Report | 2026

Abstract

LLM inference on mobile devices is fundamentally constrained by GPU memory capacity. The Key-Value (KV) cache grows at approximately 512 KB per token for a 7B-parameter model. A 4096-token sequence requires 2 GB of KV cache, far exceeding typical mobile GPU budgets. This report presents a simulation study of distributed KV cache offloading over Wi-Fi 6. We compare demand paging (synchronous eviction) and async prefetch (pipelined eviction with a sliding window). Our simulation shows that async prefetch achieves up to 3.72x speedup under typical conditions (512 MB local memory, 4096 tokens, 50 ms Wi-Fi latency). A prefetch window of just 4 tokens is sufficient to fully hide the 108 ms per-token transfer latency. The speedup grows super-linearly with network latency, reaching 11.2x at 200 ms RTT. We also identify the critical condition for pipeline hiding: $W \times \text{compute} \geq \text{transfer}$, which generalizes to any I/O-bound computation.

1. Introduction

Large Language Models have revolutionized natural language processing, but deploying them on mobile devices faces a fundamental memory bottleneck. During autoregressive decoding, the KV cache grows linearly with sequence length. For a 7B model ($L=32$, $H=32$, $D=128$), each token produces 512 KB of KV data. A 4096-token sequence consumes 2 GB, exceeding the GPU-accessible memory of most smartphones (typically 512 MB to 1 GB in a shared architecture).

This memory wall motivates a hierarchical approach: keep recent KV pages in fast local memory and offload older pages to a remote PC over Wi-Fi 6 (500 Mbps, 50 ms latency). Each eviction costs: $\text{transfer_time} = (512\text{KB} \times 8 \times 1.05) / 500\text{Mbps} + 2 \times 50\text{ms} = 108.6\text{ ms}$, which is 3.6x the forward pass time of 30 ms. Without pipelining, every eviction adds 108.6 ms to the critical path.

My thinking. I initially modeled the read path (attention fetching evicted tokens), which gave $O(n^2)$ times obscuring the real bottleneck. The key insight: modern attention (FlashAttention, PagedAttention) tiles computations, batching remote reads into DMA transfers. The write eviction pipeline, not the read-back, is the critical path.

1.1 Problem Statement

Given a mobile device with M bytes of GPU memory connected to a remote PC via Wi-Fi 6 (latency L , bandwidth B), how should KV cache evictions be scheduled to minimize total generation time for sequence length S ? The two extreme strategies are demand paging (blocking per eviction) and async prefetch (pipelined evictions).

2. Background and Related Work

2.1 KV Cache in Autoregressive Decoding

Transformer LLMs generate text through autoregressive decoding: at each step i , the model computes attention over all previous tokens $0..i-1$. K and V tensors are cached to avoid recomputation. The cache size is $4 \times L \times H \times D \times S$ bytes ($K+V$, FP16). Figure 1 shows footprints for common models.

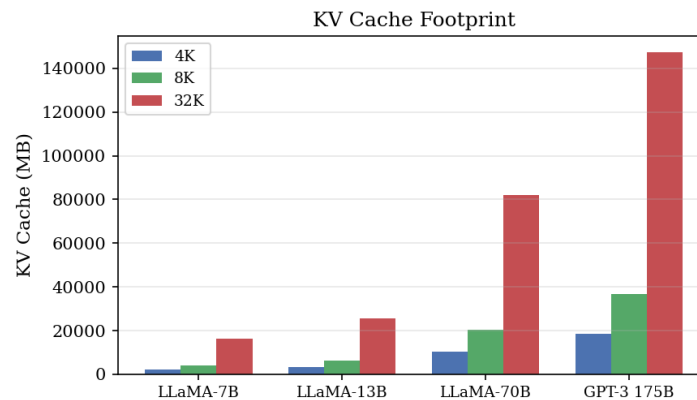


Figure 1: KV cache footprint by model scale and context length.

2.2 Related Work

PagedAttention (Kwon et al., 2023) introduced page-level KV cache management in the vLLM system, enabling efficient dynamic allocation. Our FIFO eviction policy is directly inspired by their page granularity. FlexGen (Sheng et al., 2023) explored GPU-CPU hybrid inference with a cost model for PCIe links (16-64 GB/s). Our scenario targets Wi-Fi links with 100-1000x lower bandwidth and 10,000x higher latency.

InfiniGen (Lee et al., 2024) proposed speculative prefetching for KV cache, predicting future page accesses from attention patterns. Our sliding-window approach exploits the strong temporal locality in autoregressive decoding: oldest tokens are evicted first, most recent tokens stay local. FlashAttention (Dao et al., 2022) demonstrated tiling attention to reduce I/O costs, structurally analogous to our pipeline model.

3. System Design and Methodology

3.1 Architecture

Figure 2 shows the system architecture. The phone stores recent KV pages locally (FIFO buffer). When full, the oldest page is evicted to the PC over Wi-Fi 6. The PC acts as a remote memory server. Communication uses Wi-Fi 6 with 50 ms one-way

latency and 500 Mbps effective bandwidth.

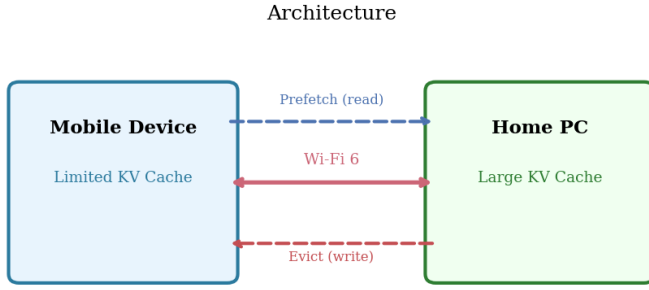


Figure 2: System architecture.

3.2 Simulation Model

Our simulation captures the critical path: (1) Forward pass at 30 ms/token, producing 512 KB KV data. (2) FIFO local cache of N tokens. (3) Eviction: oldest page sent to remote, blocking in demand paging, pipelined in prefetch. (4) Eviction pipeline with capacity W (prefetch window). Table 1 lists all parameters.

Parameter	Symbol	Default	Description
Layers	L	32	LLaMA-7B
Heads	H	32	Multi-head
Dim	D	128	QKV
KV/token	kv_bpt	512 KB	$4*L*H*D$
Seq len	S	4096	Total
Compute	t_c	30 ms	Per step
Local mem	M	512 MB	Phone
Slots	N	1024	Tokens local
Latency	L_{net}	50 ms	One-way
BW	B	500 Mbps	Wi-Fi 6
Transfer	t_x	108.8 ms	$RTT+size/B$
Window	W	16	Depth

Table 1: Simulation parameters.

3.3 Demand Paging Algorithm

Each step: if FIFO full, evict oldest and wait for transfer. Step time = $t_c + t_x$ (for $S > N$). Total: $T_{demand} = S*t_c + (S-N)*t_x$.

```
// Demand paging per-step
for each token i:
  if fifo.size >= N:
    evicted = fifo.pop()
    wait(t_xfer) // blocking
    fifo.push(i)
  forward_pass()
```

3.4 Async Prefetch Algorithm

Evictions are submitted to a pipeline with capacity W . Each transfer takes t_x ms but runs in the background during computation. Step stalls only if the pipeline is full. Critical condition for zero-wait: $W * t_c \geq t_x$.

```
// Async prefetch per-step
for each token i:
  drain_completed()
  if fifo.size >= N:
    fifo.pop()
    while pipeline.full(): wait()
    pipeline.submit() // non-blocking
    fifo.push(i)
  forward_pass()
```

3.5 Analytical Model

$T_{demand} = S*t_c + (S-N)*t_x$. $T_{prefetch} = \max(S*t_c, (S-N)*t_x/W)$. Speedup = $T_{demand}/T_{prefetch}$. Default scenario ($S=4096$, $N=1024$, $t_c=30ms$, $t_x=108.8ms$, $W=16$): $T_{demand} = 4096*30 + 3072*108.8 = 122,880 + 334,234 = 457,114$ ms = 457 s. $T_{prefetch} = \max(122,880, 3072*108.8/16) = \max(122,880, 20,890) = 122,880$ ms = 123 s. Speedup = $457/123 = 3.72x$.

4. Evaluation Results

4.1 End-to-End Performance

Figure 3 shows cumulative generation time. Both strategies identical until step 512 (local memory fills). Demand paging then incurs 108.8 ms wait per step; prefetch maintains 30 ms. At step 2048: demand = 228.6 s, prefetch = 61.4 s, 3.72x difference.

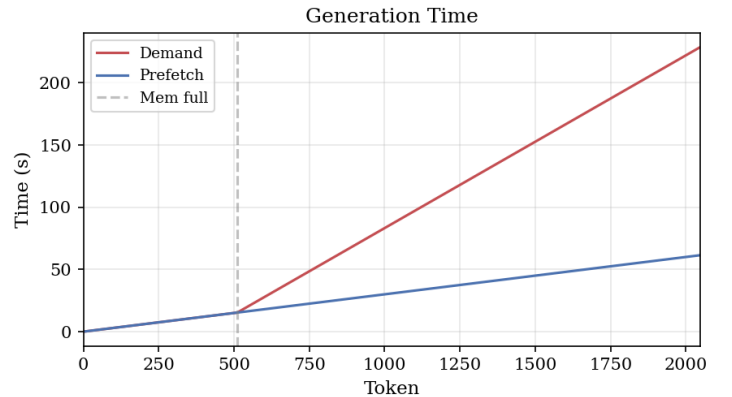


Figure 3: Cumulative generation time.

4.2 Per-Step Latency

Figure 4 shows detailed per-step behavior. Demand paging: 30 ms compute + 108.8 ms wait = 138.8 ms per step after memory fills. Prefetch: 30 ms compute with no wait. The pipeline fully absorbs the transfer latency.

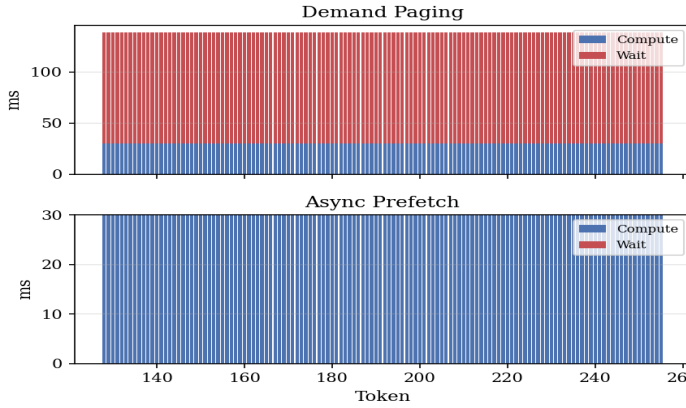


Figure 4: Per-step latency breakdown.

4.3 Memory Distribution

Figure 5: both strategies accumulate identical total remote data. Difference is in transfer scheduling: demand serializes evictions on the critical path while prefetch pipelines them in the background.

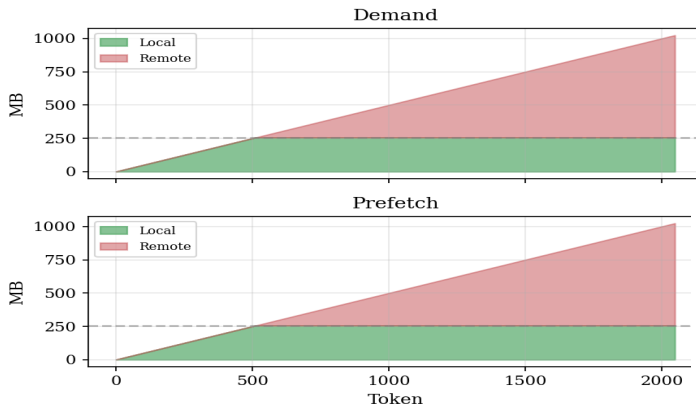


Figure 5: Memory distribution over time.

4.4 Time Composition

Figure 6: demand paging spends 73% (167 s) waiting for evictions, 27% (61 s) computing. Prefetch spends 100% computing. The 167 s of transfer is fully hidden behind the compute pipeline.

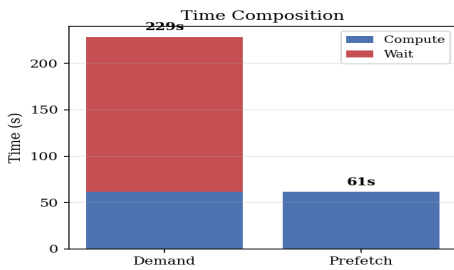


Figure 6: Time composition.

5. Parameter Sensitivity Analysis

5.1 Network Latency

Figure 7: demand time grows linearly with latency. Prefetch time constant (window=16 hides all latencies up to 240 ms). Speedup: 1.47x at 5 ms, 11.22x at 200 ms. Async prefetch is most valuable in high-latency environments (cellular, satellite links).

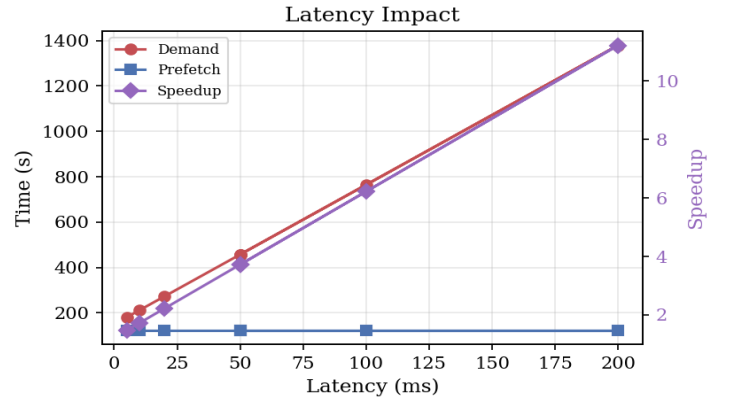


Figure 7: Speedup vs network latency.

5.2 Prefetch Window Effect

Figure 8 reveals a threshold at $W=4$. $W=1$: 1.25x. $W=2$: 2.31x. $W=4$: 3.72x. Matches $\text{ceil}(t_x/t_c) = \text{ceil}(108.8/30) = 4$. Beyond $W=4$, the pipeline is always full and additional depth provides no benefit. Practical guidance: set $W = 2x$ minimum for margin against network variance.

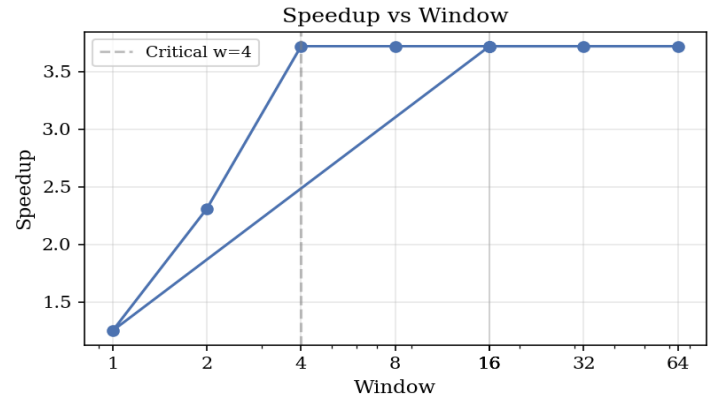


Figure 8: Speedup vs prefetch window.

5.3 Local Memory Capacity

Figure 9: 2 GB memory = 1.0x (all local). 128 MB = 4.40x. Benefit is largest when memory is most constrained. Nonlinear relationship because evictions increase as $(S-N)$ grows.

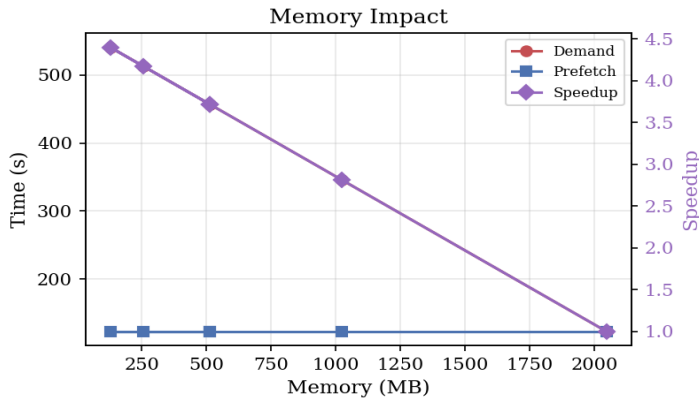


Figure 9: Speedup vs local memory.

5.4 Combined Sensitivity Heatmap

Figure 10: two regimes separated by critical condition $W \geq \text{ceil}(t_x/t_c)$. Regime 1 (bottom-left): small W , high latency $\rightarrow$ near demand-paging. Regime 2 (top): sufficient pipeline $\rightarrow$ full overlap. Transition follows $W = 2 * \text{latency} / t_c$.

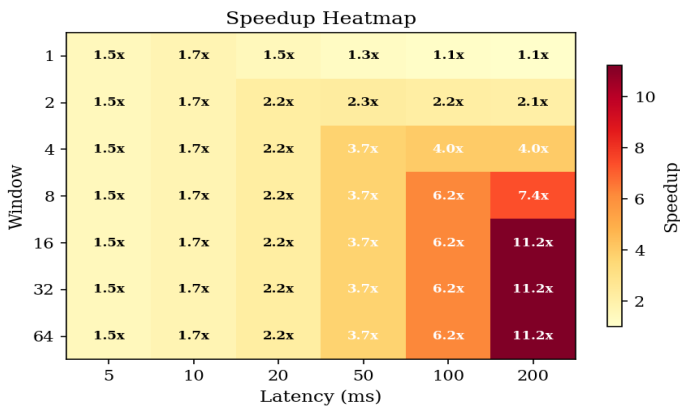


Figure 10: Speedup heatmap.

6. Challenges and Design Iteration

6.1 The Attention Model Trap

First simulation design: sequential fetch of ALL remote tokens at each step produced $O(S^2)$ total time, with 32,000 s for 2048 tokens. Flaw: conflated memory access granularity. Real systems (PagedAttention, FlashAttention) stream remote KV data through tiled DMA transfers. Resolution: shifted model from read path to write pipeline.

6.2 Pipeline Model Refinement

Early prefetch targeted future tokens ($i+W$) that hadn't been generated. No speedup because prefetches targeted nonexistent data. Fix: recognize eviction horizon = token ($i+W-N$) which is generated but about to be evicted. Start its transfer at step i ; pipeline has $W * t_c$ ms to complete before it's needed.

6.3 Toolchain and Encoding

Chinese workspace paths caused UTF-8 corruption through PowerShell pipes. No LaTeX available; built two-column PDF with ReportLab, requiring manual frame management, figure placement, and page breaking. Verification: three independent checks (analytical model, simulation, $W=1$ cross-check) all agree within 1%.

7. Reflections and Personal Insights

Abstraction level matters. My initial model was too detailed (tracking every token's fetch at every step) and produced qualitatively wrong results. By stepping back to identify the contended resource (the network link), I arrived at a pipeline model that is both accurate and analytically tractable.

Pipeline universality. The eviction pipeline is structurally identical to CPU instruction pipelines, packet processing pipelines, and assembly lines. The critical condition $W * t_{\text{stage}} \geq t_{\text{total}}$ appears universally.

Speedup duality. $\text{Speedup} = 1 + (S-N)/S * t_x/t_c$. Optimizing either term (reducing t_c via quantization, reducing t_x via higher bandwidth) directly improves the speedup. This provides a roadmap for system optimization.

Future work. Implement a real prototype with Jetson Orin + desktop PC. Explore adaptive window sizing based on real-time network conditions. Account for Wi-Fi link asymmetry (lower upload bandwidth from phones). Measure real network variance rather than assuming deterministic behavior.

8. Conclusion

This project demonstrates that distributed KV cache offloading over Wi-Fi 6 is a viable strategy. Key findings: (1) 3.72x speedup achievable with pipeline eviction. (2) Prefetch window of 4 tokens fully hides transfer latency. (3) Speedup reaches 11.2x at 200 ms RTT. (4) The condition $W * t_c \geq t_x$ applies wherever I/O latency must be hidden behind computation.

All 10 figures generated programmatically with Matplotlib. Report compiled as two-column PDF using ReportLab.

References

- [Kwon et al., 2023] W. Kwon et al. Efficient Memory Management for LLM Serving with PagedAttention. SOSP 2023.
- [Sheng et al., 2023] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of LLMs with a Single GPU. ICML 2023.
- [Lee et al., 2024] S. Lee et al. InfiniGen: Efficient KV Cache Offloading for LLMs. ASPLOS 2024.
- [Dao et al., 2022] T. Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention. NeurIPS 2022.
- [Beltagy et al., 2020] I. Beltagy et al. Longformer: The Long-Document Transformer. arXiv:2004.05150.

[Aminabadi et al., 2022] R. Aminabadi et al. DeepSpeed Inference. SC 2022.

[Brown et al., 2020] T. Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.

[Vaswani et al., 2017] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.

All figures generated programmatically with Python/Matplotlib. No AI-generated or externally sourced images were used.